Programming Principles I

Week 6 · Class 12 · Quiz 3

Quiz 3: 25 minutes

Pen only. No laptop, no phone. Write your name and student ID first.

Programming Principles I

Week 6 · Class 12

String manipulation and conversions

Length, copy, join, compare · reverse a word · text to numbers and back

```
strcpy(email, user);
strcat(email, "@ada.edu.az");
```

What Tuesday gave us

1. A string is an array of `char` that ends with `'\0'`
2. `char city[20] = "Baku";` keeps room for 19 letters
3. `%s` prints a string; `scanf("%19s", city)` reads one word
4. A loop can walk along a string until it meets `'\0'`
5. `'7' - '0'` is the number `7`

Recap: what is on the screen?

PREDICT

A loop walks along "Baku".

What does it print?

```
int main(void) {
    char city[20] = "Baku";
    int i = 0;
    while (city[i] != '\0') {
        i++;
    }
    printf("length = %d\n", i);
    return 0;
}
```

Recap: what is on the screen?

PREDICT

A loop walks along `"Baku"`.

What does it print?

```c
int main(void) {
    char city[20] = "Baku";
    int i = 0;
    while (city[i] != '\0') {
        i++;
    }
    printf("length = %d\n", i);
    return 0;
}
```

ANSWER

`length = 4`

The loop stops at `'\0'`, after B, a, k, u.

Today's roadmap

1. `<string.h>`: **length** and **copy**
2. **Join** two strings
3. **Compare** two strings
4. By hand: **reverse** and **palindrome**
5. **Text to numbers**, and back

> Tuesday you wrote the loops yourself. Today C does most of them for you.

1 Length and copy

The string toolbox: #include <string.h>

A toolbox for strings

Add one line at the top: `#include <string.h>`

`strlen(s)`: how many characters

`strcpy(to, from)`: copy a string

`strcat(to, more)`: add to the end

`strcmp(a, b)`: compare two strings

strlen: the length

```c
int main(void) {
    char city[20] = "Baku";
    printf("strlen: %d\n", (int) strlen(city));
    printf("sizeof: %d\n", (int) sizeof(city));
    return 0;
}
```

Output

```
strlen: 4
sizeof: 20
```

`strlen`: letters before `'\0'`.

`sizeof`: all 20 boxes.

strlen is not sizeof

box	0	1	2	3	4	5 to 19
`city`	B	a	k	u	`\0`	empty

`strlen(city)` counts B, a, k, u: **4**. `sizeof(city)` counts every box: **20**.

A loop up to strlen

```c
int main(void) {
    char word[20] = "Baku";
    int n = strlen(word);
    for (int i = 0; i < n; i++) {
        printf("%c ", word[i]);
    }
    printf("\n");
    return 0;
}
```

Output

```
B a k u
```

n is 4: `i` goes 0, 1, 2, 3.

Careful: give the array a new name

CAREFUL

Can we put `"Leyla"` into `name` with `=`?

```
int main(void) {
    char name[20];
    name = "Leyla";
    printf("%s\n", name);
    return 0;
}
```

No: = does not work for arrays

```c
int main(void) {
    char name[20];
    name = "Leyla";
    printf("%s\n", name);
    return 0;
}
```

gcc says

```
error: assignment
to expression with
array type
```

= works at the declaration only.

strcpy: copy into the array

```c
int main(void) {
    char name[20];
    strcpy(name, "Leyla");
    printf("%s\n", name);
    strcpy(name, "Murad");
    printf("%s\n", name);
    return 0;
}
```

Output

```
Leyla
Murad
```

`strcpy(to, from)`: the same order as `to = from`. The old name is gone.

2 Join

strcat adds to the end

strcat: add to the end

```c
int main(void) {
    char full[30] = "Ada";
    strcat(full, " ");
    strcat(full, "Lovelace");
    printf("%s\n", full);
    printf("%d characters\n", (int) strlen(full));
    return 0;
}
```

Output

```
Ada Lovelace
12 characters
```

`full` has 30 boxes: room for all.

full, line by line

after the line	full	strlen
`char full[30] = "Ada";`	Ada	3
`strcat(full, " ");`	Ada	4
`strcat(full, "Lovelace");`	Ada Lovelace	12

`strcat` writes over the old `'\0'` and puts a new one at the end.

Careful: is there room?

CAREFUL

`small` has 6 boxes.

We add `" there!"`.

What happens?

```
int main(void) {
    char small[6] = "Hi";
    strcat(small, " there!");
    printf("%s\n", small);
    return 0;
}
```

gcc warns: it overflows

```c
int main(void) {
    char small[6] = "Hi";
    strcat(small, " there!");
    printf("%s\n", small);
    return 0;
}
```

gcc -Wall says

```
warning: writing
8 bytes into a
region of size 6
overflows the
destination
```

> `"Hi there!"` needs 10 boxes.

And yet it prints

```c
int main(void) {
    char small[6] = "Hi";
    strcat(small, " there!");
    printf("%s\n", small);
    return 0;
}
```

Output

```
Hi there!
```

It wrote past the end of `small`.

A wrong program can look right.

The rule for strcpy and strcat

> **The array must have room for the result, plus one.**

`"Hi there!"`: 9 characters + `'\0'` = 10 boxes.

C does not check. You count.

Make the array bigger than you need: `char full[100];`

3

Compare

Never ==, always strcmp

Predict: you type yes

PREDICT

The program asks "Continue?".

You type `yes`.

What does it print?

```
int main(void) {
    char answer[10];
    printf("Continue? ");
    scanf("%9s", answer);
    if (answer == "yes") {
        printf("OK, going on\n");
    } else {
        printf("Stopping\n");
    }
    return 0;
}
```

It says Stopping

```c
int main(void) {
    char answer[10];
    printf("Continue? ");
    scanf("%9s", answer);
    if (answer == "yes") {
        printf("OK, going on\n");
    } else {
        printf("Stopping\n");
    }
    return 0;
}
```

Output

```
Continue? yes
Stopping
```

The letters were never compared.

gcc saw it

```c
int main(void) {
    char answer[10];
    printf("Continue? ");
    scanf("%9s", answer);
    if (answer == "yes") {
        printf("OK, going on\n");
    } else {
        printf("Stopping\n");
    }
    return 0;
}
```

gcc -Wall says

```
warning: comparison
with string literal
results in
unspecified behavior
```

== does not compare letters

For strings, == asks: "the same array?"

`answer` and `"yes"` are two different arrays.

So `==` is false, whatever you typed.

To compare the letters: `strcmp(answer, "yes") == 0`

strcmp: three answers

```c
int main(void) {
    printf("%d\n", strcmp("apple", "banana"));
    printf("%d\n", strcmp("banana", "apple"));
    printf("%d\n", strcmp("apple", "apple"));
    printf("%d\n", strcmp("Zoo", "apple"));
    return 0;
}
```

Output

```
-1
1
0
-1
```

What strcmp(a, b) says

result	meaning	example
negative	a comes first	`strcmp("apple", "banana")`
0	equal	`strcmp("apple", "apple")`
positive	a comes later	`strcmp("banana", "apple")`

Only the sign matters. Capitals come before small letters: `"Zoo"` is before `"apple"` (`'Z'` is 90, `'a'` is 97).

A password check

```c
int main(void) {
    char word[20];
    printf("Password: ");
    scanf("%19s", word);
    if (strcmp(word, "ada2026") == 0) {
        printf("Welcome\n");
    } else {
        printf("Wrong password\n");
    }
    return 0;
}
```

Two runs

```
Password: ada2026
Welcome

Password: ADA2026
Wrong password
```

Equal means `== 0`.

Which name comes first?

```c
int main(void) {
    char a[20], b[20];
    printf("Two names: ");
    scanf("%19s %19s", a, b);
    if (strcmp(a, b) < 0) {
        printf("First: %s\n", a);
    } else {
        printf("First: %s\n", b);
    }
    return 0;
}
```

Output

```
Two names: Murad Aysel
First: Aysel
```

`< 0`: a comes first.

4 By hand

Reverse a word, test a palindrome

Reverse: swap from both ends

```
void reverse(char s[]) {
    int i = 0;
    int j = strlen(s) - 1;
    while (i < j) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}
```

`i` from the left, `j` from the right.

The swap from class 10.

No size parameter: `'\0'` marks the end.

Reverse "hello", step by step

```
void reverse(char s[]) {
    int i = 0;
    int j = strlen(s) - 1;
    while (i < j) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}
```

i	j	swap	word
-	-		hello
0	**4**	**h, o**	**oellh**

`j` starts at `strlen - 1` = 4: the last letter.

Reverse "hello", step by step

```
void reverse(char s[]) {
    int i = 0;
    int j = strlen(s) - 1;
    while (i < j) {
        char temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}
```

i	j	swap	word
-	-		hello
0	4	h, o	oellh
1	3	e, l	olleh
2	**2**	**stop: i < j is false**	**olleh**

The middle letter `l` stays where it is. Output: `olleh`.

A palindrome reads the same both ways

```
int is_palindrome(char s[]) {
    int n = strlen(s);
    for (int i = 0; i < n / 2; i++) {
        if (s[i] != s[n - 1 - i]) {
            return 0;
        }
    }
    return 1;
}
```

`s[i]` and its partner `s[n - 1 - i]`.

One pair differs: `return 0`.

level, kayak, hello, abba

```
int main(void) {
    check("level");
    check("kayak");
    check("hello");
    check("abba");
    return 0;
}
```

Output

```
level: yes
kayak: yes
hello: no
abba: yes
```

A string in quotes can go straight to a function.

5 Text and numbers

"2026" is not 2026

"2026" is not 2026

> **Four characters are not one number.**

`"2026"`: the characters `'2'` `'0'` `'2'` `'6'`, then `'\0'`

`2026`: one `int` you can add to, compare, multiply

Input from a file or a form often arrives as text.

Text to number, by hand

```c
int main(void) {
    char s[10] = "2026";
    int n = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        n = n * 10 + (s[i] - '0');
    }
    printf("n = %d\n", n);
    printf("n + 1 = %d\n", n + 1);
    return 0;
}
```

`s[i] - '0'`: the digit, as on Tuesday.

`n * 10`: one place to the left.

"2026" becomes 2026

```
char s[10] = "2026";
int n = 0;
for (int i = 0; s[i] != '\0'; i++) {
    n = n * 10 + (s[i] - '0');
}
```

i	s[i]	digit	n
0	'2'	2	0 * 10 + 2 = **2**
1	**'0'**	**0**	**2 * 10 + 0 = 20**

"2026" becomes 2026

```
char s[10] = "2026";
int n = 0;
for (int i = 0; s[i] != '\0'; i++) {
    n = n * 10 + (s[i] - '0');
}
```

i	s[i]	digit	n
0	'2'	2	0 * 10 + 2 = **2**
1	'0'	0	2 * 10 + 0 = **20**
2	'2'	2	20 * 10 + 2 = **202**
3	**'6'**	**6**	**202 * 10 + 6 = 2026**

Then `s[4]` is `'\0'`: the loop stops.

It is a real number now

```c
int main(void) {
    char s[10] = "2026";
    int n = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        n = n * 10 + (s[i] - '0');
    }
    printf("n = %d\n", n);
    printf("n + 1 = %d\n", n + 1);
    return 0;
}
```

Output

```
n = 2026
n + 1 = 2027
```

atoi does the same job

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("%d\n", atoi("2026") + 1);
    printf("%d\n", atoi("-45"));
    printf("%d\n", atoi("12abc"));
    printf("%d\n", atoi("abc"));
    return 0;
}
```

Output

```
2027
-45
12
0
```

`atoi` reads a sign and the digits, and stops at the first other character. No number at the start: **0**.

Number to text, by hand

```c
int n = 472;
char s[12];
int len = 0;
while (n > 0) {
    s[len] = '0' + n % 10;
    len++;
    n = n / 10;
}
s[len] = '\0';
printf("backwards: %s\n", s);
```

Output

```
backwards: 274
```

`n % 10`: the last digit.

`n / 10`: drop it.

472, digit by digit

```
while (n > 0) {
    s[len] = '0' + n % 10;
    len++;
    n = n / 10;
}
```

n	n % 10	s	n / 10
472	2	"2"	47
47	7	"27"	4
4	**4**	**"274"**	**0**

The last digit comes out first. So the string is backwards: reverse it.

Then reverse it

```
for (int i = 0; i < len / 2; i++) {
    char temp = s[i];
    s[i] = s[len - 1 - i];
    s[len - 1 - i] = temp;
}
printf("text: %s\n", s);
```

Output

```
text: 472
```

sprintf: printf into a string

```c
int main(void) {
    char s[20];
    sprintf(s, "%d", 472);
    printf("text: %s\n", s);
    printf("length: %d\n", (int) strlen(s));

    char msg[40];
    snprintf(msg, sizeof(msg),
             "%s scored %d", "Aysel", 95);
    printf("%s\n", msg);
    return 0;
}
```

Output

```
text: 472
length: 3
Aysel scored 95
```

`snprintf` never writes past the size.

Text and numbers: two ways each

job	by hand	with a function
text to `int`	`n = n * 10 + (s[i] - '0')`	`atoi(s)` (`<stdlib.h>`)
`int` to text	`n % 10`, `n / 10`, reverse	`sprintf(s, "%d", n)`

6 Your turn

Two short tasks on paper

Your turn: digit_sum

YOUR TURN

Write `int digit_sum(char s[])`.

It adds the digits of a string.

`digit_sum("2026")` is 2 + 0 + 2 + 6 = 10.

digit_sum: one answer

```
int digit_sum(char s[]) {
    int sum = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        sum += s[i] - '0';
    }
    return sum;
}

int main(void) {
    printf("%d\n", digit_sum("2026"));
    printf("%d\n", digit_sum("999"));
    return 0;
}
```

Output

```
10
27
```

Tuesday's loop to `'\0'`.

Your turn: an e-mail address

YOUR TURN

Read a user name, like `aysel`.

Print `aysel@ada.edu.az`.

Use `strcpy` and `strcat`. How big must the array be?

E-mail: one answer

```c
int main(void) {
    char user[20];
    char email[40];
    printf("User name: ");
    scanf("%19s", user);
    strcpy(email, user);
    strcat(email, "@ada.edu.az");
    printf("%s\n", email);
    return 0;
}
```

Output

```
User name: aysel
aysel@ada.edu.az
```

`email` has 40 boxes: name + 11 + 1.

Today in one slide

1. `strlen`: the letters before `'\0'`; not `sizeof`
2. `strcpy` to put a string in an array; never `=`
3. `strcat` adds to the end: count the room
4. Equal strings: `strcmp(a, b) == 0`; never `==`
5. Text to number: `n * 10 + digit` or `atoi`; back: `sprintf`

Before Tuesday

1. **Knowledge check 12** on Blackboard: today's slides
2. **Quiz 4: Fri 23 October**, classes 11-12: strings
3. **Project 2**: announced this week, on Blackboard
4. **Midterm**: week of 26 October, classes 1 to 14
5. Today's code: Blackboard, Week 6 module, **Class 12 code**

Next class: two-dimensional arrays. Tables of numbers.